深度审计与架构升级报告：基于动态时间依赖的城市绿色物流配送调度

资深运筹优化算法审计师

1 第一章 审计背景与项目初始化状态声明

作为资深运筹优化算法审计师，本次任务的核心目标是对第十八届“华中杯”A 题《城市绿色物流配送调度》的现有算法架构进行深度审查，并为即将展开的第三问（动态事件响应）提供具备坚实数学基础、前沿文献支撑与工程可行性的架构重构方案。

在执行初始化指令时，系统尝试直接读取提供的 GitHub 仓库链接。然而，网络访问记录显示该 GitHub 仓库当前处于无法访问或未公开状态 [1, 2]。因此，本报告的现状审计与技术路线推演完全基于所提供的核心项目上传文档（包括第一问与第二问的模型求解总结文档以及第三问初始化提示词文档）所反映的事实构建 [3, 3]。基于这些事实建立的客观认知足以支撑对现有系统现状的精准锁定，并为后续优化提供可靠依据。

当前项目的宏观背景是一个高度复杂的车辆路径规划问题(Vehicle Routing Problem, VRP)。由于现实城市场景的复杂性，该问题不仅包含了异构车队（Heterogeneous Fleet）的资源分配，还涉及严格的“绿色区域”(Green Zone) 政策限制，以及基于时变路网（Time-Dependent Road Network）和随机车速分布的动态行驶时长与能耗计算 [3, 4]。第一问与第二问作为静态确定性问题（Static and Deterministic VRP），已经通过自适应大邻域搜索（Adaptive Large Neighborhood Search, ALNS）算法框架得出了基准解 [3, 3]。第三问则要求在静态基准方案的执行过程中，应对订单取消、新增、地址变更等突发事件。这标志着问题本质从静态 VRP 正式跨越至带时间窗的动态车辆路径问题（Dynamic Vehicle Routing Problem with Time Windows, DVRP-TW），对算法的实时响应能力、物理状态跟踪精度以及求解稳定性提出了极高的要求 [5, 6, 7]。

2 第二章 现状审计与物理核心约束校准

在构建动态响应架构之前，必须对项目当前已完成的技术栈、物理建模逻辑及第二问的基准指标进行严密的校验。第二问的结果不仅是静态阶段的最终交付物，更是第三问动态调度的参考系与“冻结状态”的基础 [3]。

2.1 2.1 拓扑结构与绿色区域硬约束

本项目的空间拓扑并未采用连续的地理信息系统（GIS）道路网络，而是基于节点间的距离矩阵（通过 `customer_id` 映射虚拟服务节点）构建离散图模型 [3]。这种抽象方式简化了路径几何层面的计算，但也要求算法在节点序列层面严格把控空间限制。

“绿色区域”政策是本项目的核心硬约束（Red Line）。根据设定，绿色区域是一个以城市中心 $(0,0)$ 为圆心、半径为 10 公里的圆形地理围栏。而统一的物流配送中心（Depot）位于坐标 $(20,20)$ 处，处于绿色区域之外 [3, 3]。在每天的 08:00 至 16:00（即基于午夜 00:00 计算的第 480 分钟至 960 分钟区间内，$T_{\text{green}} = [480, 960)$），燃油车辆（F1、F2、F3 车型）被严格禁止服务位于该区域内的任何客户节点 [3]。

审计结果显示，在当前的基准方案中，算法面临着 2169 个原始订单、98 个客户节点以及 1 个配送中心的庞大规模。通过合理的订单聚合与拆分逻辑（推荐方案 `DEFAULT_SPLIT`），系统生成了 148 个虚拟服务节点，其中包含 19 个位于绿色区域内的节点，总需求达到约 35,971 千克和 104 立方米 [3]。第二问的最终调度方案实现了**零政策冲突**，表明当前的构造型启发式与 ALNS 算子在处理这一带有时间窗的地理围栏硬约束时具备绝对的有效性 [3]。在第三问的动态架构中，这一硬约束的检查机制不仅不能被削弱，反而需要嵌入到实时状态机中，以防止因动态拥堵导致的燃油车被动违规滞留。

2.2 2.2 目标函数与异构车队物理动态

系统的总体目标是最小化总配送成本（C_{total}），该目标函数聚合了四项子成本：固定成本、能耗成本、碳排放成本以及时间窗惩罚成本 [3, 3]。

表 1: 当前基准模型成本结构与物理参数校验

成本维度	**计算逻辑与审计发现**	**第二问基准表现**
固定成本 (C_{fixed})	按实际调用的物理车辆数计费(400/辆)。模型支持物理车辆的多趟复用（Multi-Trip），这是非常贴近工程现实的设计。	调用 10 辆 E1 与 35 辆 F1，固成本 18,000 [3]。
能耗与碳排 ($C_{\text{energy}}, C_{\text{carbon}}$)	燃油单价 7.61/L，电力 1.64/kWh。碳排放系数分别为 2.547 kg/L 和 0.501 kg/kWh。	成本核算逻辑一致。
时变速度与 FIFO 准则	行驶速度依拥堵、畅通、正常等时间段变化。算法正确实现了先进先出（FIFO）逻辑。	未发现时空倒流现象。
软时间窗惩罚 (C_{tw})	早到等待惩罚 20/小时，迟到惩罚 50/小时。作为软约束处理。	存在 12 个迟到点，最大延129.44 分钟 [3]。

在能耗模型的审计中，发现了一个极其关键且高度严谨的数学处理：系统对速度变量（v）应用了**詹森不等式修正 (Jensen's Correction)** [3, 8]。由于能耗通常与速度的平方成正比，而车辆在路段上的速度服从特定均值 μ 与方差 σ^2 的概率分布。根据概率论，平方的期望 $E[v^2]$ 并不等于期望的平方 μ^2，而是 $E[v^2] = \mu^2 + \sigma^2$ [3, 8]。当前架构准确捕捉了这一随机方差对能耗的非线性放大效应，并在计算基于载荷率（λ）的能量消耗时进行了整合。这一机制使得模型在物理规律上站得住脚，也为第三问中因订单取消导致的载荷率动态变化提供了精确的能耗修正基础。

综上所述，第二问最终在总成本 49,239.78 的水平上实现了全局收敛 [3]。针对服务质量更为敏感的对照组（`p500` 变体），虽然最大延迟时间显著缩短，但总成本上升至 50,770.72 [3]。对于即将展开的第三问，必须明确以 49,239.78 的基准结果作为动态重规划的参考锚点。

3 第三章 第三问（动态事件响应）的理论重构与前沿文献溯源

随着新订单插入、已有订单取消以及地址变更等动态事件的引入，第一、二问中的离线计算（Offline Routing）范式将彻底失效。此时，系统演变为一个在线或动态车辆路径问题（DVRP）[9, 10]。根据国际顶尖学术期刊（如《Transportation Science》）的研究分类，动态路径规划的核心难点在于“信息质量”与“信息演变”的不可预知性 [11]。

3.1 3.1 动态车辆路径问题（DVRP）的技术演进

在学术界的长期发展中，DVRP 已经从早期的简单贪婪调度演化为复杂的马尔可夫决策过程与元启发式优化框架。Pillac 等人的综述详尽分析了动态与随机车辆路径问题的分类学 [9]。在完全确定且静态的 VRP 中，所有信息在车辆驶离车场前已被完全掌握。而在本项目的第三问中，部分客户请求的到达时间是动态的，且路网状态可能会随事件发生突变，这属于“动态且确定性”（Dynamic and Deterministic）问题演化至“动态且随机”（Dynamic and Stochastic）问题的边界地带 [12]。

由于传统 VRP 的 NP-Hard 属性，当面临实时事件时，重新从零开始求解整个问题所需的时间往往超出事件响应的时间窗口要求。此外，如 Bertsimas 与 Simchi-Levi 所指出，静态模型中用于评估解质量的单纯成本最小化目标，在动态滚动时域模型中往往不再适用，因为动态系统不仅需要关注成本，更需要关注“稳健性”（Robustness）以及对未知事件的吸收能力 [13]。

3.2 3.2 动态决策触发机制与滚动时域（Rolling Horizon）理论

针对第三问，业界最为主流且经过验证的数学框架是**滚动时域优化策略（Rolling Horizon Optimization Strategy）** [3, 14]。该策略的本质是将连续的物理时间轴离散化为一系列的决策时间窗。系统的优化触发机制分为两种：时间驱动（Time-driven，固定周期触发）与事件驱动（Event-driven，新事件发生时立即触发）[10]。由于本题目的情境更倾向于应对离散的突发订单或取消行为，采用**事件驱动的滚动时域策略**最为契合。

滚动时域的核心逻辑在于：每当一个新事件在时间 t_{event} 发生时，算法并不废弃所有既定计划，而是将时间轴划分为“已发生且不可逆的过去”和“待优化的未来”。在时间点 t_{event}，算法生成一个问题状态的快照，将所有已经执行的服务和正在路段上行驶的车辆状态“冻结”，仅仅将尚未执行的任务节点放入一个“未分配池”（Unassigned Pool）中，随后对这个规模缩小后的子问题进行快速重优化 [15]。

这不仅极大地缩减了问题规模（Dimensionality Reduction），使得 ALNS 等元启发式算法能够在数秒内给出可行解，更重要的是，它符合真实的物理配送规律：车辆不可能在半路瞬间掉头，已经卸货的订单也不可能被收回。

4 第四章 动态问题状态空间映射与滚动架构设计

在落实上述理论时，需要对现有代码架构进行大幅度且谨慎的改动。我们建议重构原有的 `Solution` 状态机，建立一个专用于第三问的动态状态抽象层（`DynamicProblemState`）。

4.1 4.1 状态空间（State Space）的数学定义

当事件在时间 t_{event} 触发时，整个物流网络的状态空间 $S(t_{\text{event}})$ 可被严密地划分为四个子集 [3, 16]：

1. **冻结轨迹（$\mathcal{R}_{\text{frozen}}$）：**
   对于所有其服务开始时间 $t_{\text{service},i} \leq t_{\text{event}}$ 的客户节点 i，其所在的路径前缀被严格锁定。这部分轨迹所产生的固定成本、行驶能耗与碳排放、以及已经发生的时间窗惩罚，将作为常数项 $C_{\text{accumulated}}$ 计入最终成本，不再参与后续的搜索优化迭代。

2. **动态在途状态锁定（In-Transit State Lock）：**
   对于车辆 k，如果其在时间 $t_{\text{dep},i}$ 从节点 i 驶出，但在事件发生时 t_{event} 尚未抵达下一个计划节点 j（即 $t_{\text{dep},i} < t_{\text{event}} < t_{\text{arr},j}$），车辆的物理位置必须通过时间插值进行映射。在实际工程处理中，我们不可改变当前路段 (i, j) 的目的地，因此节点 j 必须作为该车辆未来的“必达锚点”（Locked Target）[17]。车辆 k 参与后续重新分配任务的起点即为节点 j，其可用开始时间被更新为 $t_{\text{arr},j}$ 加服务时间。

3. **未分配请求池（$\mathcal{P}_{\text{unassigned}}$）：**
   所有尚未被锁定服务或作为必达锚点的节点，从原有的路径序列中剥离，连同因动态事件产生的新增订单节点，一同汇入未分配请求池。如果是取消订单事件，该订单对应的虚拟节点直接从网络中永久剔除。

4. **动态环境矩阵（$\mathcal{M}_{\text{dynamic}}$）：**
   实时同步由于地址修改导致更新的距离矩阵，或是由于突发交通状况带来的路网时变速度曲线更迭。

4.2 4.2 子问题的在线生成

通过上述映射，原本庞大的全局 VRP 被坍缩为一个起点各异的非标准 VRP 子问题。每个正在运营的物理车辆都被赋予了一个新的虚拟起步状态，即其在 t_{event} 之后的第一个可用时间和可用地点。这种在线子问题的动态生成逻辑，是确保启发式算法不会陷入对已发生事实无效优化的前提，同时也保证了不会有任何一辆物理车辆被安排执行空间上断裂的行程 [3, 15]。

5 第五章 目标函数的重定义：稳定性度量与多目标惩罚机制

第三问所面临的最核心的工程困境在于：如果每次出现一个微小的动态事件，算法都为了追求绝对的成本最小化而对全局进行“大洗牌”，将会导致极端的物理执行灾难。

5.1 5.1 稳定性的现实意义与研究进展

正如 Transportation Science 和各种运筹学顶级期刊中的文献所强调的，在动态车辆路径问题中，调度方案的“稳定性”（Stability / Consistency）是一项至关重要的指标 [18, 19, 20]。如果在配送中途频繁变更司机接下来的所有停靠站点，不仅会增加司机的认知负荷与操作错误

率，更可能因为货物在车厢内部的堆叠顺序（LIFO，后进先出原则）而导致物理上无法卸货。虽然本项目未严格设定三维装箱约束，但在算法审计的专业视角下，减少对原基准计划的破坏度（Spatiotemporal Displacement）应当作为一种高级别的服务质量保障要求被引入目标函数 [18]。

在提供给第三问的初始化指令中，明确提出了需要输出对比基准方案的“路线变更规模”（Scale of route changes）等稳定性指标 [3]。因此，必须将单目标的成本最小化问题，改造为兼顾成本与稳定性的多目标优化问题（Multi-objective Optimization），或通过将稳定性指标转化为惩罚项融合进单一目标函数中 [21]。

5.2 5.2 稳定性惩罚项（Stability Penalty）的数学构造

我们建议在目标函数中引入两种层级的稳定性惩罚机制，以引导算法在进行路径重构时，优先保留原有的优良结构。

机制一：车辆重分配惩罚（节点级稳定性）
该惩罚旨在抑制客户订单在不同物理车辆之间的频繁转移。设 $x_{i,k}^{\text{old}} \in \{0,1\}$ 为第二问基准方案中，节点 i 被分配给车辆 k 的布尔变量。设 $x_{i,k}^{\text{new}}$ 为当前动态优化方案的分配状态。

$$C_{\text{stability_node}} = \omega_1 \sum_{i \in \mathcal{P}_{\text{unassigned}}} \sum_{k \in \mathcal{V}_{\text{active}}} |x_{i,k}^{\text{new}} - x_{i,k}^{\text{old}}|$$

当一个未完成的订单被硬性塞给一辆原本不负责该区域的新车时，该函数产生固定惩罚 ω_1。这一惩罚权重可以通过实验进行微调，以在成本与司机路线熟悉度之间寻找平衡 [21, 22]。

机制二：边破坏惩罚（序列级稳定性）
比节点重分配更严格的是对访问序列的破坏。哪怕节点仍在同一辆车上，如果原来的连续访问顺序被打乱，依然会产生隐性成本。引入指示变量 $y_{i,j}^{\text{old}} \in \{0,1\}$，当基准方案中存在直接从节点 i 驶向节点 j 的有向边时，其值为 1。

$$C_{\text{stability_edge}} = \omega_2 \sum_{(i,j) \in E} \max(0, y_{i,j}^{\text{old}} - y_{i,j}^{\text{new}})$$

该公式有效地衡量了“原始连边被拆断”的数量。通过叠加这两种惩罚，第三问在每一轮次迭代时的局部评估目标函数 $F_{\text{obj}}^{(m)}$ 被扩展为：

$$F_{\text{obj}}^{(m)} = \left[C_{\text{fixed}}^{(m)} + C_{\text{energy}}^{(m)} + C_{\text{carbon}}^{(m)} + C_{\text{tw}}^{(m)}\right]_{\text{future}} + C_{\text{stability_node}} + C_{\text{stability_edge}}$$

这一多目标架构确保了，如果新路线仅能节约微不足道的 2 元能耗成本，但却需要拆散 5 个原有的连续拜访边，算法在计算综合代价后将拒绝这一调整，从而最大程度维护路线结构的稳定性 [21, 23]。

6 第六章 自适应大邻域搜索（ALNS）在动态场景下的深度重构

项目当前依赖的 ALNS 算法表现优异，但其设计初衷是用于长时间的离线批量求解 [3, 24]。面对动态事件，如果每次响应都要运行数万次迭代，耗时可能长达数十分钟，这在实际运营中

是不可接受的。必须针对 DVRP 场景开发一套 “轻量级、高针对性” 的 ALNS 算子组合（Light ALNS）[3]。

表 2: ALNS 动态场景下的算子重构对比

算子类型	**静态 ALNS 传统实现**	**动态 DVRP 重构要求（Light ALNS）**
破坏算子 (Destroy)	随机移除 (Random)、邵氏移除 (Shaw/Relatedness)、最差移除 (Worst)。对全图节点无差别攻击 [24, 25]。	必须加入 “冻结保护” 机制，严禁触碰 $\mathcal{R}_{\text{frozen}}$。引入**动态破坏算子**，根据在途车辆当前地理坐标而非起始车场坐标重新计算相关性。
修复算子 (Repair)	贪心插入 (Greedy)、Regret-k 插入。往往倾向于遍历全解空间寻找极小值 [24, 26]。	优先采用**基于遗憾值的启发式插入 (Regret-k)** 以快速满足硬约束。引入**局部贪心快速修复**以便在极短时间内生成初始可行解 [26]。
温度控制 (SA)	高起始温度，极慢的冷却率，旨在充分探索复杂地形。	起始温度 T_0 应与事件扰动程度挂钩。单点取消使用低温局部搜索；突发批量新订单使用高温进行有限范围内的拓扑重组 [27]。

6.1 6.1 动态破坏算子（Dynamic Destroy Operators）的专门设计

在处理动态问题时，文献证明专门设计的启发式算子能显著提升求解速度与质量 [25, 28]。

1. **历史路径截断算子（Historical Route Destruction）：**
   针对因突发新订单插入导致某辆车后续行程全部违背软时间窗，或导致燃油车 F1 被迫拖延至 T_{green} 禁行时段的极端恶劣情况。此算子不按传统方式零散地移除客户节点，而是直接将该问题车辆在当前锚点之后的所有序列 “一刀切” 剥离，释放回未分配池，并迫使其他车队（尤其是 E1 电动车）进行支援 [3]。

2. **动态相关性移除（Dynamic Shaw Removal）：**
   传统的 Shaw 算子依赖于空间与时间的接近度。但在动态场景下，空间接近度必须动态映射车辆在 t_{event} 的实时地理坐标，而非原本的客户与客户间静态矩阵。这将帮助算法迅速识别出因为路线拖延而发生空间交叠的不同车辆，并剥离它们相互冲突的客户进行重新洗牌 [25]。

6.2 6.2 动态修复算子（Dynamic Repair Operators）与硬约束满足

1. **带绿色区域感知的 Regret-k 修复（Green-Aware Regret Repair）：**
   Regret 算子的核心逻辑是计算将某客户插入最佳位置与次佳位置的成本差额。差额越大，说明该节点越 “别无选择”，必须优先插入 [26]。在动态环境下，对于身处 10 公里绿色区域内的客户，其唯一合法的运载工具在 T_{green} 时段内仅限于纯电动车型（E1），或者必须

被推迟到 16:00 以后由燃油车执行 [3]。这一严格的政策限制将导致这类节点的 Regret 值极大。算法会优先将区域内的动态节点锁定到 E1 车队的路径中，从而在极少迭代内达成“零政策冲突”的硬指标。

2. **快速可行性修复（Local Deterministic Repair）：**
当接收到一个微小的地址变更或时间窗微调事件时，无需启动重型 ALNS。只需将变更节点从原位置抠出，并通过最小插入成本算子（Minimum Insertion Cost Repair）直接计算其在新路线上的最佳增量位置 [3, 26]。这种近乎 $\mathcal{O}(N)$ 复杂度的操作为系统带来了极佳的在线响应性能。

7 第七章 物理规律的动态校验：时间窗、FIFO 与能耗模型的联动

本项目的另一个难点是，任何针对路线序列的微小改动，不仅会改变地理距离，更会通过时变路网和载重量改变引发多米诺骨牌式的物理状态级联变化。第三问的架构重构必须妥善处理这些联动机制。

7.1 7.1 时变速度的 FIFO 积分动态推演

旅行时间 TT 并非一个静态矩阵查询值，而是基于车辆驶入路段的具体时间点 t_{dep}，对时变速度分布曲线 $v(t)$ 积分得到的求解过程：

如果在动态事件响应中，我们向车辆序列中强行插入了一个新节点，该车辆随后抵达所有下游节点的时间将被延迟。这种延迟可能导致车辆原本在自由流（Free-flow）时段通过的路段，不幸落入拥堵（Congested）时段 [4]。因此，修复算子每进行一次位置评估，就必须触发一次正向的 FIFO 时间流推演引擎，重算后续所有节点的时间窗惩罚成本 C_{tw}。算法底层必须确保：无论如何调整，$t_1 < t_2$ 必定推导出 $t_1 + TT(t_1) \leq t_2 + TT(t_2)$。一旦出现逆向违规，算法需自动在此刻注入虚拟等待时间，以维系物理时间的单向流动特性 [4, 29]。

7.2 7.2 动态载荷率（Load Ratio）对 Jensen 能耗模型的影响

第二问的成果中完美融合了带有 Jensen 修正的二次速度能耗方程，这是一个值得保持的亮点 [3, 8]。车辆每公里的能耗率不仅与随机车速的方差有关，还受到当前装载状态参数 λ（当前载重 / 最大载重）的影响。

在第三问中，必须特别处理“订单取消”事件带来的隐性收益。当某节点被动态撤销时，该车辆不仅缩短了绕路距离，更关键的是车厢内永久性地减少了该订单所对应的质量。这就意味着在剩余的整个行程中，车辆的负载系数 λ 整体下调，其加速和维持匀速所需的物理做功成比例减少 [3, 8]。这就要求在代码实现上，`DynamicEvaluator` 模块不能仅仅使用累加的方法计算能耗变化，而是必须对被影响车辆的 λ 序列数组进行全局向下更新，重新应用经过 Jensen 修正的能耗积分公式，准确计算卸负荷带来的长尾红利。反之，新订单的插入将全局推高 λ 导致下游路段能耗激增，此时稳定性目标和能耗目标将形成相互制衡。

8 第八章 第三问工程架构重写与实施路线图

从理论映射到实际的 Python 项目代码，需要建立一套严密的工程执行路径，并且严格遵守前两问定下的“红线”规范 [3]。

8.1 8.1 核心代码模块扩展计划

现有的 `alns.py` 和 `scheduler.py` 面向批量处理。需增加以下外围控制模块，以最小的侵入性实现动态功能：

1. **事件截获与状态管理模块（`state_manager.py`）：**
   输入：静态运行产生的 `outputs/problem2/recommendation.json` 以及发生的动态事件向量。
   处理逻辑：通过比对 t_{event} 与 `stop_schedule.csv` 中的时间戳，将解集一分为二，生成只包含只读数据的 $\mathcal{R}_{\text{frozen}}$ 对象，以及包含所有游离实体的 $\mathcal{P}_{\text{unassigned}}$ 列表 [3]。

2. **动态目标评估器（`dynamic_objective.py`）：**
   继承原有成本评估器类，覆盖计算逻辑。增加关于稳定性偏离的测量：加载被截获解集的拓扑网络，利用前述公式并行计算 $C_{\text{stability_node}}$ 与 $C_{\text{stability_edge}}$ 的罚值，合并进局部最优判断 [21]。

3. **在线调度网关（`online_gateway.py`）：**
   管理对 ALNS 优化器的重复调用。当面临极小扰动时（例如单一包裹重量微调），甚至可以选择直接跳过 ALNS 迭代，单纯运行一次 Greedy 修补并返回。当面临全局性扰动（例如大面积封路）时，动态拔高起始温度并运行固定时长（如限制 15 秒计算预算）的启发式搜索 [30]。

8.2 8.2 工程纪律与输出规范

根据初始化指令的强制要求，动态响应阶段的工作成果绝不能覆写第一问或第二问的输出目录。所有经过滚动时域更新的路径排班表应当写入专门的 `outputs/problem3/` 目录下 [3]。在生成最终的新版 `route_summary.csv` 时，模块必须执行硬性的逻辑校验：

- **空间非叠加性校验：**任何一辆实体车（Physical Vehicle ID）不能在同一时空维度执行相互冲突的路径片段 [3]。

- **载荷上限校验：**即便是在应对多点订单积压的情况下，新生成的虚拟节点体积和重量也不可突破异构车队的物理容积上限。

- **绝对合规性校验：**F 族燃油车的行驶记录日志将被遍历检查。一旦发现在 $T_{\text{green}} \in [480, 960)$ 区间内存在任何位于半径 10 公里坐标圈内的轨迹，该解作废重算 [3]。

9 结语

基于对当前项目文档的严密审查，可以确认，算法体系在处理带有时变属性、非线性物理能耗以及地理围栏特征的大型异构车辆路径规划中，已经具备了出色的离线寻优能力与数学建模深度。

面对第三问向动态带时间窗车辆路径问题（DVRP-TW）的演进，系统无需推翻第一、二问底层的物理机制，而是必须在宏观调度架构上完成一次关键的范式跃迁。通过引入事件驱动的滚动时域策略（Rolling Horizon），我们将时间线一切为二，利用动态状态映射锁定已发生事实与车辆物理在途位置。为了避免短视的重规划带来现实执行的混乱，本方案通过构建多目标的节点-边稳定性惩罚模型，有效地引导 ALNS 算子在局部进行温和修复而非破坏性重构。结合专门裁剪的带有绿色区域强约束感知的 Regret-k 与历史截断算子，这套重构方案有充分的理论依据在毫秒级内应对动态扰动，提供既满足物理连贯性，又在数学意义上高度优化的新一代绿色物流调配规划方案。